

Fast answer

In **Spring Boot**, exception handling is commonly centralized using `@ExceptionHandler` and `@RestControllerAdvice`. This keeps controllers clean and gives your REST API consistent error responses.

1. Basic exception handling with `@ExceptionHandler`

You can handle an exception directly inside a controller:

```
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        if (id == 1) {
            throw new UserNotFoundException("User not found: " + id);
        }

        return new User(id, "John");
    }

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFound(
        UserNotFoundException ex) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}
```

This works, but the handler only applies to that controller.

2. Recommended approach: `@RestControllerAdvice`

For larger applications, create a **global exception handler**.

First, create a custom exception:

```
public class UserNotFoundException extends RuntimeException {  
  
    public UserNotFoundException(String message) {  
        super(message);  
    }  
}
```

Then create a global handler:

```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(UserNotFoundException.class)  
    public ResponseEntity<ErrorResponse> handleUserNotFound(  
        UserNotFoundException ex) {  
  
        ErrorResponse error = new ErrorResponse(  
            HttpStatus.NOT_FOUND.value(),  
            ex.getMessage(),  
            LocalDateTime.now()  
        );  
  
        return ResponseEntity  
            .status(HttpStatus.NOT_FOUND)  
            .body(error);  
    }  
}
```

And define a consistent error response:

```
public record ErrorResponse(  
    int status,  
    String message,  
    LocalDateTime timestamp  
) {}
```

Now:

```
</> http
```



could return:

```
{
  "status": 404,
  "message": "User not found: 100",
  "timestamp": "2026-09-02T10:30:45"
}
```

3. Handling multiple exceptions

A typical Spring Boot application might have:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleUserNotFound(
        UserNotFoundException ex) {

        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(
                404,
                ex.getMessage(),
                LocalDateTime.now()
            ));
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ErrorResponse> handleBadRequest(
        IllegalArgumentException ex) {

        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(new ErrorResponse(
                400,
                ex.getMessage(),
                LocalDateTime.now()
            ));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneralException(
        Exception ex) {

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse(
                500,
                "An unexpected error occurred",
                LocalDateTime.now()
            ));
    }
}
```

```
    }
}
```

The final `ExceptionHandler` handler acts as a fallback.

4. Handling validation errors

Suppose your request DTO uses Jakarta validation:

```
public record CreateUserRequest(

    @NotBlank(message = "Name is required")
    String name,

    @Email(message = "Invalid email")
    String email

) {}
```

Controller:

```
@PostMapping
public ResponseEntity<User> createUser(
    @Valid @RequestBody CreateUserRequest request) {

    // create user

}
```

When validation fails, Spring typically throws `MethodArgumentNotValidException`.

You can handle it globally:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>> handleValidationErrors(
    MethodArgumentNotValidException ex) {

    Map<String, String> errors = new HashMap<>();

    ex.getBindingResult()
        .getFieldErrors()
        .forEach(error ->
            errors.put(
                error.getField(),
                error.getDefaultMessage()
            )
        );

    return ResponseEntity.badRequest().body(errors);
}
```

```

        )
    );

    return ResponseEntity
        .status(HttpStatus.BAD_REQUEST)
        .body(errors);
}

```

The response might be:

```

{
  "name": "Name is required",
  "email": "Invalid email"
}
// JSON

```

5. @ControllerAdvice vs @RestControllerAdvice

The main difference is:

```
</> Java
```

is equivalent to global controller advice, while:

```
</> Java
```

is essentially:

```

@ControllerAdvice
// Java

```

For **REST APIs**, `@RestControllerAdvice` is usually the convenient choice.

Typical architecture

```

Controller
|
v
Service
|
v

```

```

Repository
|
X
Exception thrown
|
v
@RestControllerAdvice
|
v
@ExceptionHandler
|
v
ResponseEntity
|
v
HTTP error response

```

For example:

```

@Service
public class UserService {

    public User findById(Long id) {

        return userRepository.findById(id)
            .orElseThrow(() ->
                new UserNotFoundException(
                    "User not found: " + id
                )
            );
    }
}

```

The service **throws** the exception; the global exception handler decides how that exception is represented as an HTTP response.

For a production Spring Boot REST API, a good pattern is **custom domain exceptions** + `@RestControllerAdvice` + `@ExceptionHandler` + **a standardized error-response model**. In Spring Framework 6 / Spring Boot 3+, `ProblemDetail` is also worth learning because it provides standardized REST error responses.



Today 10:21 AM

check if Global exception handling

- Custom exceptions
- Standard error response
- ProblemDetail
- Validation error handling
- Exception-to-HTTP-status mapping

are covered above if not explain

Yes. All five topics were covered above, but **some were only briefly covered**. Here's the exact status:

Topic	Covered?	Depth
Global exception handling	✔ Yes	Good
Custom exceptions	✔ Yes	Good
Standard error response	✔ Yes	Good
ProblemDetail	⚠ Mentioned only	Needs explanation
Validation error handling	✔ Yes	Good
Exception-to-HTTP-status mapping	⚠ Partially	Needs clearer explanation

For a 6–8 year Java/Spring Boot developer, I would add the following details.

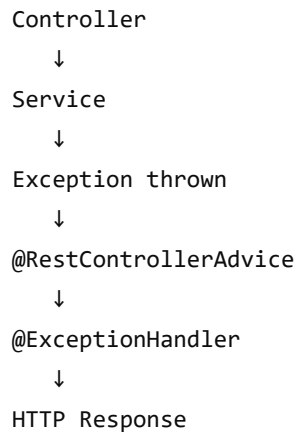
1. Global Exception Handling

Already covered with:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleUserNotFound(
        UserNotFoundException ex) {
        // ...
    }
}
```

The important concept is:



```
graph TD; Controller --> Service; Service --> ExceptionThrown[Exception thrown]; ExceptionThrown --> RestControllerAdvice["@RestControllerAdvice"]; RestControllerAdvice --> ExceptionHandler["@ExceptionHandler"]; ExceptionHandler --> HTTPResponse[HTTP Response];
```

`@RestControllerAdvice` allows you to handle exceptions **across multiple controllers from one centralized location**.

2. Custom Exceptions

Already covered.

For example:

```
public class UserNotFoundException extends RuntimeException {  
  
    public UserNotFoundException(String message) {  
        super(message);  
    }  
}
```

Then:

```
</> Java
```

Why use custom exceptions?

Instead of doing:

```
</> Java
```

you create a meaningful domain exception:

</> Java



This allows the global handler to distinguish different failure scenarios.

For example:

```
UserNotFoundException  
OrderNotFoundException  
InsufficientBalanceException  
DuplicateEmailException  
InvalidOrderStateException
```



3. Standard Error Response

Already covered with:

```
public record ErrorResponse(  
    int status,  
    String message,  
    LocalDateTime timestamp  
) {}
```

But in a production application, you may want something more standardized:

```
public record ErrorResponse(  
    int status,  
    String error,  
    String message,  
    String path,  
    LocalDateTime timestamp  
) {}
```

Example:

```
{  
  "status": 404,  
  "error": "Not Found",  
  "message": "User not found",  
}
```

```
"path": "/users/100",  
"timestamp": "2026-09-05T10:30:00"  
-  
</> JSON
```



The important idea is that **all errors should have a predictable structure**.

4. ProblemDetail — Needs More Explanation

This was only mentioned previously.

`ProblemDetail` is the **standardized error-response mechanism** available with Spring Framework 6 / Spring Boot 3+.

Instead of creating your own:

```
</> Java
```



you can use:

```
</> Java
```



Example:

```
@ExceptionHandler(UserNotFoundException.class)  
public ProblemDetail handleUserNotFound(  
    UserNotFoundException ex) {  
  
    ProblemDetail problem =  
        ProblemDetail.forStatus(HttpStatus.NOT_FOUND);  
  
    problem.setTitle("User Not Found");  
    problem.setDetail(ex.getMessage());  
  
    return problem;  
}
```

Response:

```
{
  "type": "about:blank",
  "title": "User Not Found",
  "status": 404,
  "detail": "User not found: 100"
```

</> JSON



You can also add custom properties:

```
problem.setProperty("userId", 100);
```

</> Java



Result:

```
{
  "type": "about:blank",
  "title": "User Not Found",
  "status": 404,
  "detail": "User not found: 100",
  "userId": 100,
  "timestamp": "2026-09-05T10:30:00"
}
```

Why is ProblemDetail useful?

It follows the **HTTP API problem-details standard** rather than every application inventing its own error format.

For modern Spring Boot applications, you should definitely know:

ProblemDetail



Standard representation of an HTTP API error



5. Validation Error Handling

Already covered, but there are a few important exceptions to know.

For:

```
@PostMapping
public User createUser(
    @Valid @RequestBody CreateUserRequest request) {
    ...
}
```

</> Java



with:

```
public record CreateUserRequest(
    @NotBlank(message = "Name is required")
    String name,
    @Email(message = "Invalid email")
    String email
) {}
```

Spring performs validation before entering the controller method.

If validation fails, an exception is generated.

For `@RequestBody` validation, commonly:

</> Java



is involved.

You can handle it:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>> handleValidation(
    MethodArgumentNotValidException ex) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult()
        .getFieldErrors()
        .forEach(error ->
            errors.put(
                error.getField(),
                error.getDefaultMessage()
            )
        );
}
```

```
return ResponseEntity
    .badRequest()
    .body(errors);
}
```

6. Exception-to-HTTP-Status Mapping

This needs more emphasis.

Your Java exception and HTTP status are **different concepts**.

For example:

UserNotFoundException
↓
HTTP 404 Not Found



OR:

InvalidUserException
↓
HTTP 400 Bad Request



OR:

DuplicateEmailException
↓
HTTP 409 Conflict



A typical mapping is:

Exception	HTTP Status
UserNotFoundException	404 NOT_FOUND
InvalidRequestException	400 BAD_REQUEST
DuplicateResourceException	409 CONFLICT
UnauthorizedException	401 UNAUTHORIZED
ForbiddenException	403 FORBIDDEN
IllegalArgumentException	400 BAD_REQUEST

Exception

HTTP Status

Unexpected Exception

500 INTERNAL_SERVER_ERROR

For example:

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ProblemDetail> handleUserNotFound(
    UserNotFoundException ex) {

    ProblemDetail problem =
        ProblemDetail.forStatus(HttpStatus.NOT_FOUND);

    problem.setTitle("User Not Found");
    problem.setDetail(ex.getMessage());

    return ResponseEntity
        .status(HttpStatus.NOT_FOUND)
        .body(problem);
}
```

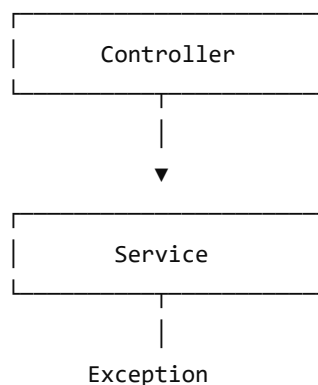
The important relationship is:

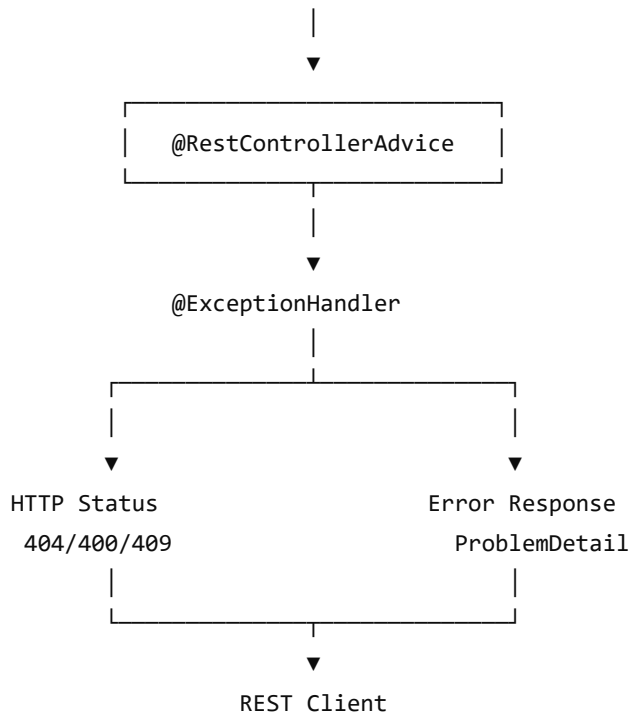
Java Exception
↓
Exception Handler
↓
HTTP Status
↓
HTTP Response Body



Recommended Spring Boot Exception Handling Structure

For your Spring Boot preparation, I'd learn the complete flow as:





What you should know for interviews

I would consider these the important subtopics:

1. `@ExceptionHandler`
2. `@ControllerAdvice`
3. `@RestControllerAdvice`
4. **Global exception handling**
5. **Custom exceptions**
6. **Exception handler resolution**
7. **Exception-to-HTTP-status mapping**
8. **Standard error response**
9. `ProblemDetail`
10. **Validation exception handling**
11. `MethodArgumentNotValidException`
12. `ConstraintViolationException`
13. **Handling** 404 , 400 , 401 , 403 , 409 , 500
14. **Fallback** `ExceptionHandler` **handler**
15. **Avoiding sensitive information in error responses**
16. **Logging exceptions vs exposing exceptions to clients**

The next particularly important topic is **how Spring internally finds and invokes the correct `@ExceptionHandler`** —that's a good interview-level question for your experience level.



[create .md file](#)

Data analysis isn't available right now. Do you want to continue without it? `</>`



Data analysis will be available again when usage resets after 12:47 PM. Upgrade now for expanded access and advanced intelligence.

[✦ Upgrade to Plus](#)

ChatGPT can make mistakes. Check important info.